

HORUS: OSINT News Ingestion and Processing Pipeline

Internship Report (CC3051) | 2025/26

Ricardo Castro Figueiredo
Skill & Reach

January 2026

Department of Computer Science
Faculty of Sciences, University of Porto

Abstract

This report describes the design and implementation of HORUS, an automated OSINT news ingestion and processing pipeline developed during a 16-week internship at Skill & Reach. The system addresses the challenge of collecting, enriching, and delivering cybersecurity-relevant news from diverse open-source feeds. The pipeline integrates RSS parsing, web scraping with headless rendering, and LLM-based enrichment to automatically classify articles by sector, criticality, and relevance to the company's service portfolio. Articles are indexed in Elasticsearch with vector embeddings for deduplication and similarity search. A FastAPI backend provides authenticated access to the processed data, while a React-based Control Center enables source management, article exploration, and automated email report generation. Key technical contributions include a multi-source ingestion system handling structured RSS/Atom feeds and unstructured websites, an LLM-powered classification pipeline using OpenAI's API, and an embedding-based article selection mechanism for personalized reports. This work demonstrates practical applications of AI-assisted content processing in the cybersecurity intelligence domain.

Keywords: OSINT, RSS/Atom feeds, OpenAI API, Cybersecurity, Web Scraping, LLM Enrichment, ETL pipeline

Contents

1	Introduction	5
1.1	Context and Motivation	5
1.2	Objectives and Scope	5
1.3	Deliverables and Expected Outcomes	5
2	State of the Art	6
2.1	Open-Source Intelligence (OSINT) Pipelines	6
2.2	News Ingestion and RSS/HTML Scraping	6
2.3	Text Enrichment Using LLMs and Embeddings	6
2.4	Search and Storage: Elasticsearch	6
3	System Requirements and Design	7
3.1	Functional Requirements	7
3.2	Non-functional Requirements	7
3.3	High-level Architecture	8
3.3.1	Components Overview	8
3.3.2	Data Model	8
3.4	Deployment and Runtime Environment	9
3.5	Interfaces and APIs	9
4	Description of Work	11
4.1	Approach	11
4.1.1	Design decisions and rationale	11
4.1.2	Choice of technologies and libraries	12
4.2	Ingestion	12
4.2.1	RSS feed parser and feed handling	12
4.2.2	Scraping subsystem, headless rendering and listing discovery	13
4.3	Processing and Enrichment	14
4.3.1	Sanitization and Content Extraction	14
4.3.2	LLM-Based Classification and Article Detection	14
4.3.3	Metadata Enrichment and Abstract Generation	15
4.3.4	Embedding Computation, Deduplication, and Related-Article Linking	16
4.4	Storage and Indexing	16
4.4.1	Index Design and Mapping Configuration	16
4.4.2	Bulk Ingestion and Error Handling Strategy	17

4.5	Backend API Service	17
4.6	Frontend Control Center	18
4.7	Email Delivery	19
4.7.1	Article Selection for Email Reports	19
4.7.2	Email Assembly and Delivery Mechanism	19
5	Conclusion and Future Work	20
5.1	Future Work	20
	Acknowledgements	21

1 Introduction

1.1 Context and Motivation

In the cybersecurity domain, staying informed about emerging threats, vulnerabilities, and industry developments is critical for both service providers and their clients. Skill & Reach, a cybersecurity consultancy specializing in NIS 2, DORA compliance, and cyber risk management, identified a need to systematically monitor and distribute relevant intelligence to stakeholders. Manual curation of news from multiple sources proved time-consuming and inconsistent, creating an opportunity for automation.

This internship aimed to develop HORUS, an end-to-end pipeline for automated Open-Source Intelligence (OSINT) news ingestion, classification, and delivery. The system needed to aggregate content from RSS and Atom feeds and websites, enrich articles with metadata using AI, and provide both a web interface for exploration and automated email reports tailored to client interests.

1.2 Objectives and Scope

The primary objective was to implement a production-ready pipeline capable of:

- **Ingesting** news from multiple sources (RSS/Atom feeds and web scraping)
- **Processing** articles through LLM-based classification for sector, criticality, and relevance
- **Storing** enriched articles in Elasticsearch with vector embeddings for deduplication
- **Delivering** content via automated email reports
- **Managing** sources and users through a React-based Control Center and authenticated API endpoints

The scope included developing three main components: a Python-based ingestion pipeline, a FastAPI backend, and a React.js frontend. The system was designed to run continuously in Docker containers, processing articles every 10 minutes.

1.3 Deliverables and Expected Outcomes

The internship delivered a fully functional system comprising:

- A containerized pipeline with RSS parsing, headless scraping, and LLM enrichment
- An Elasticsearch index schema optimized for news article storage and retrieval
- A FastAPI backend with JWT authentication and comprehensive article/source management
- A React Control Center for article exploration, source configuration, and email generation
- Automated email report functionality with article selection and scheduling

2 State of the Art

2.1 Open-Source Intelligence (OSINT) Pipelines

OSINT pipelines automate the collection, processing, and analysis of publicly available information. Modern OSINT systems typically follow an Extract-Transform-Load (ETL) architecture, separating data acquisition from enrichment and storage. Tools like Maltego and SpiderFoot provide commercial OSINT capabilities, while open-source alternatives like TheHive and MISP focus on threat intelligence sharing. These systems often lack domain-specific classification and struggle with unstructured web content beyond RSS feeds.

2.2 News Ingestion and RSS/HTML Scraping

RSS/Atom feed parsing remains the primary method for structured news ingestion. The `feedparser` library provides robust parsing with format normalization across RSS and Atom feeds. For unstructured sources, headless browsers enable JavaScript-rendered content extraction. Playwright offers reliable automation across Chromium, Firefox, and WebKit engines. BeautifulSoup4 handles HTML parsing and DOM traversal for content extraction.

2.3 Text Enrichment Using LLMs and Embeddings

Large Language Models (LLMs) enable automated text classification, summarization, and tagging. OpenAI's GPT models provide structured outputs via function calling and JSON schema validation, suitable for extracting sectors, criticality levels, and keywords from articles. The `openai` Python client supports these capabilities.

Vector embeddings transform text into dense numerical representations for similarity search and deduplication. The `sentence-transformers` library provides pre-trained models like `all-MiniLM-L6-v2`, offering efficient encoding with 384-dimensional vectors. Cosine similarity between embeddings enables duplicate detection and related article recommendations without exact string matching.

2.4 Search and Storage: Elasticsearch

Elasticsearch provides distributed full-text search with JSON document storage. Its inverted index enables millisecond-latency queries across millions of documents, while the bulk API supports efficient batch indexing. Alternative solutions like PostgreSQL exist, but Elasticsearch's maturity and integrated search capabilities suit news pipeline requirements.

3 System Requirements and Design

3.1 Functional Requirements

The system must support the following capabilities:

- **Multi-source ingestion:** Fetch articles from RSS/Atom feeds and scrape websites with JavaScript rendering support
- **Extract metadata:** Extract and calculate metadata like title, content, publication date, language, and estimated read time from the article
- **Automated classification and enrichment:** Classify articles by sectors, criticality, tags, and relevance to company services, and enrich articles with an abstract
- **Deduplication:** Identify and filter duplicate or highly similar articles using content-based hashing and locally computed vector embeddings
- **Source management:** Enable managers to add and configure news sources with confidence scoring
- **User authentication:** Provide role-based access control (admin/user) with email/password based authentication
- **Article exploration:** Browse, filter, and view detailed article information through a web interface
- **Email delivery:** Generate and schedule customized email reports with automatic article selection

3.2 Non-functional Requirements

Performance: The pipeline iterates through every source every 10 minutes. The system aims to complete single article processing within 30 seconds, including ingestion, enrichment, and storage, but this is not strictly enforced. Timeouts are implemented for headless navigation (`HEADLESS_NAV_TIMEOUT_MS`) and LLM calls (`LLM_TIMEOUT_SEC`).

Reliability: The system handles transient network failures and malformed feeds through retry mechanisms and error logging. Failed articles do not block subsequent processing. LLM-based classification is optional; the pipeline falls back to heuristic methods when LLM is disabled or errors occur.

Scalability: The architecture supports growth to hundreds of sources. The pipeline can also scale horizontally by running multiple workers with different source subsets.

Security: Most API endpoints require authentication, with the following exceptions: `/auth/login` (login endpoint), and `/healthz` (health check). Endpoints accessed by the pipeline require authentication by API key, and endpoints accessed by the Control Center UI require JWT authentication. Sensitive credentials (OpenAI API keys, JWT secrets, email API keys) are externalized via environment variables. User passwords are hashed with `bcrypt`.

Maintainability: Code in the pipeline is modular with clear separation between ingestion (`ingest/`), processing (`process/`), storage (`store/`), and delivery (`deliver/`). Structured logging provides per-cycle and per-source statistics for diagnostics.

3.3 High-level Architecture

3.3.1 Components Overview

The system follows a layered ETL architecture deployed as four Docker services communicating via HTTP and shared volumes.

- **Pipeline Worker** (`apps/pipeline/`): Scheduled Python process (APScheduler) executing 10-minute ingestion cycles
 - *Ingest*: RSS parser (`rss.py`) normalizes feeds; scraping subsystem (`scraping/`) uses Playwright for headless rendering and listing discovery with configurable timeout (`HEADLESS_NAV_TIMEOUT_MS`)
 - *Process*: Sanitizes HTML, classifies with OpenAI GPT when configured (`llm_classifier.py`), computes local vector embeddings (`embeddings.py`) using the Sentence Transformers library with MiniLM-L6-v2 (384 dimensions) by default, estimates read time, and finds related articles
 - *Store*: Bulk-indexes enriched articles to Elasticsearch via API endpoint
- **FastAPI Backend** (`apps/api/`): REST API (port 8000) with SQLite database
 - Authentication: permanent API key (used by pipeline) or JWT token with 10-hour expiry (used by control center)
 - Articles: Search/filter with pagination, timeseries aggregation, auto-selection via embeddings
 - Sources: CRUD operations with ignore lists and status tracking
 - Users: Admin-only management with bcrypt hashing
 - Emails: Draft creation, scheduling, and delivery via Resend API
- **Control Center UI** (`apps/control-center/`): React SPA (port 5173, Vite dev server)
 - Dashboard: Article statistics with time-series charts, bar charts, and tables
 - Articles page: Filterable and sortable table with faceted search, custom filters and detail views
 - Sources page: Configuration interface with add/edit/disable controls
 - Email builder: Email composer with manual and automatic article selection
- **Elasticsearch**: Document store (port 9200) with 512MB heap, single-node mode

3.3.2 Data Model

Article (Elasticsearch `news` index): Documents with `_id` (URL hash), `title`, `url`, `source`, `abstract` (LLM summary), `content`, `sectors` (array from 19 predefined categories), `criticality` (low/medium/high/critical), `tags` (LLM keywords), `embedding` (384-dim float array), `published_at` (ISO8601), `confidence` (0-5), `offer` (matched service from 7 options), `estimated_read_time` (e.g., "4 min read"), and `related` (array of similar article IDs).

Source (SQLite `sources`): Configuration with `id`, `name`, `url`, `input_type` (rss/scrape), `confidence` (0-5), `enabled` (boolean), `ignore_urls` (JSON array), `last_status`, `last_checked_at`, timestamps.

User (SQLite users): `id`, `email`, `password_hash`, `role` (user/admin), timestamps.

Email (SQLite emails): `id`, `subject`, `recipient`, `article_urls` (JSON), `status` (draft/scheduled/sent/failed), `scheduled_for`, timestamps.

3.4 Deployment and Runtime Environment

The system runs via `compose.yml` defining four services with health checks and restart policies. Elasticsearch uses a persistent volume (`elasticsearch_data`) for indices. The API stores SQLite databases in `sources_data` volume. Services communicate via Docker's internal network (`api:8000`, `elasticsearch:9200`).

Key environment variables: `LLM_API_KEY` (OpenAI), `JWT_SECRET` (token signing), `RESEND_API_KEY` (email delivery), `API_STATIC_KEY` (pipeline authentication).

3.5 Interfaces and APIs

The FastAPI backend exposes RESTful JSON endpoints at the root path:

Public endpoints (no authentication required):

- `POST /auth/login`: Returns JWT token (`access_token`, `token_type`: "bearer") with 10-hour expiry (600 minutes default)
- `GET /healthz`: Health check endpoint for monitoring

Articles (requires JWT or static API key):

- `GET /articles`: Paginated list with filters (`q` for full-text search, `sectors[]`, `tags[]`, `categories[]`, `criticality`, `published_from`, `published_to`, `source`, `confidence_min`, `confidence_max`, `offer`, `page`, `per_page`, `sort`)
- `GET /articles/by-url?url=...`: Fetch article by exact URL
- `GET /articles/by-ids`: Fetch multiple articles by ID list
- `POST /articles/bulk`: Bulk fetch articles by URL list
- `GET /articles/timeseries`: Aggregate counts by hour or day intervals
- `GET /articles/select`: Auto-select diverse articles using embedding-based deduplication
- `GET /articles/suggest`: Typeahead suggestions for facet values
- `GET /articles/facet-counts`: Available filter values with document counts

Sources (requires JWT or static API key):

- `GET /sources`: List all sources
- `POST /sources`: Create new source
- `GET /sources/{id}`: Fetch source by ID
- `PATCH /sources/{id}`: Update source configuration

- DELETE /sources/{id}: Remove source

Emails (requires JWT or static API key):

- POST /email/schedule-email: Schedule personalized email report
- POST /email/schedule-broadcast: Schedule broadcast to multiple recipients
- GET /email/sent: List sent and scheduled emails
- GET /email/sent/{id}/preview: Preview email HTML content
- PATCH /email/sent/{id}/status: Update email status (cancel scheduled)
- POST /email/check-duplicates: Validate article selection for duplicates

Users (requires JWT from admin user, static API key rejected):

- GET /admin/users: List all users
- POST /admin/users: Create user with email, password, and role
- PATCH /admin/users/{id}: Update user details
- DELETE /admin/users/{id}: Delete user account

Authentication is handled via **Authorization: Bearer <token>** (JWT) or **X-API-Key: <key>**. The static API key is accepted only by routes depending on `allow_api_key_or_jwt`, enabling pipeline worker access without user credentials. Admin endpoints (`/admin/*`) explicitly require a JWT from an admin-role user and reject static API keys.

Response codes used: 200 (success), 201 (created), 204 (no content), 400 (validation errors), 401 (authentication failure), 403 (authorization failure), 404 (not found), 409 (conflict), 500 (internal errors), 502 (Elasticsearch backend failures). Errors return JSON `{"detail": "message"}`.

4 Description of Work

4.1 Approach

The system implements a modular pipeline architecture that separates the core concerns of ingestion, processing, storage, and delivery. This separation of responsibilities enables independent scaling, testing, and maintenance of each component. The overarching data flow is unidirectional: ingestion collects raw content from configured sources; processing sanitizes, classifies, and enriches this content; storage persists the enriched articles for retrieval; and delivery mechanisms expose the processed intelligence via an API, a web interface, and scheduled email reports. All runtime components are containerized and orchestrated via Docker Compose, which ensures a consistent and reproducible environment from local development to production deployment. This design prioritizes operational reliability through health checks and restart policies, idempotency via robust deduplication, and configurability through environment variables.

4.1.1 Design decisions and rationale

Several key design decisions underpin the system’s architecture, each chosen to address specific requirements.

Modular, Decoupled Pipeline: The ingestion and enrichment logic is isolated within a dedicated `pipeline` service, separate from the `api` and `control-center`. This allows the computationally intensive and potentially long-running tasks (web scraping, LLM calls) to operate asynchronously without blocking user-facing services. It also simplifies independent scaling and enables the pipeline to be run as a scheduled batch job.

Unified Search: Elasticsearch was selected as the primary persistence layer for articles. It serves as a traditional inverted index for fast full-text search and aggregations.

Layered Deduplication Strategy: To ensure idempotent ingestion, a two-stage deduplication process is employed. First, a canonical URL and content hash catch exact and near-exact duplicates during ingestion. Second, a similarity check using vector embeddings (with a configurable threshold) identifies semantically similar content before indexing. This approach minimizes redundant notifications and storage.

Pluggable LLM Provider Abstraction: LLM interactions for classification and summarization are abstracted behind a provider interface, configured via environment variables (`LLM_PROVIDER`, `LLM_MODEL`). This design provides flexibility to switch between different AI service providers (e.g., Google, Anthropic, local models) and control costs through configurable rate limits and timeouts.

Configuration-Driven Headless Scraping: The use of a headless browser for JavaScript-rendered content is made optional via the `HEADLESS_ENABLED` flag. This allows administrators to disable this resource-intensive operation when scraping static sites or during debugging, improving performance and reducing complexity when not required.

API-First with JWT and Static Key Auth: The system adopts an API-first design, with the FastAPI backend serving as the single source of truth for all data operations. Authentication is designed for human and machine clients: user sessions from the Control Center use short-lived JWT tokens, and the internal pipeline service uses a permanent static API key.

4.1.2 Choice of technologies and libraries

The technology stack was selected for its maturity, developer ergonomics, and alignment with the system’s asynchronous and data-intensive nature.

Backend (Python API): The core data processing and API layer is built with **FastAPI** for its high performance, automatic OpenAPI documentation, and native support for asynchronous request handling. It is served by the ASGI server **uvicorn**. Data modeling and interaction with the lightweight SQLite metadata database (for users, sources, and email schedules) is handled by **SQLModel**, which combines Pydantic validation with SQLAlchemy ORM patterns. The HTTP client **httpx** is used for its async capabilities and HTTP/2 support in the pipeline.

Processing: The ingestion pipeline relies on **feedparser** for RSS/Atom parsing and **playwright** for headless browser automation. HTML parsing and content extraction use **beautifulsoup4**. LLM operations utilize the official **openai** Python client. Embeddings are generated locally using the **sentence-transformers** library (defaulting to the **all-MiniLM-L6-v2** model). Job scheduling within the pipeline is managed by **APScheduler**, and resilience is bolstered by **tenacity** for retry logic.

Frontend (Control Center): The user interface is a Single Page Application built with **React** and **TypeScript**, bundled with **Vite**. Routing is handled by **react-router-dom**, charts are rendered using **recharts**, and date manipulation uses **date-fns**.

Orchestration: All services are containerized using Docker and orchestrated via a single **compose.yml** file. This includes the custom-built pipeline, API, and frontend services, alongside an official **elasticsearch** image. This setup defines internal networks, persistent volumes for data, and health checks.

Supporting Libraries: Security functions use **python-jose** for JWT handling and **passlib** for password hashing (bcrypt). Email delivery is integrated via the **resend** Python client. Content sanitization can leverage libraries like **bleach**, and read-time estimation uses the **readtime** package.

4.2 Ingestion

The ingestion subsystem is the component responsible for collecting raw content from all configured sources. It is designed to handle two primary input types: structured RSS/Atom feeds and unstructured web pages requiring scraping. Operating within the scheduled pipeline worker, this module transforms disparate data formats into a normalized internal representation. The process is incorporated with retry mechanisms, configurable timeouts, and comprehensive logging.

4.2.1 RSS feed parser and feed handling

The ingestion of structured content is managed by a dedicated RSS/Atom parser, which uses the **feedparser** library (v6.0.12). The system supports two methods for sourcing feed URLs: a static list provided via environment variables (**EnvSourceProvider**) and a dynamic list fetched from the internal API’s **/sources** endpoint (**ApiSourceProvider**). The latter is the primary method in production.

Each feed is fetched using the **httpx** HTTP client with appropriate timeouts and retries via the **tenacity** library. Upon successful retrieval, the raw XML is parsed by **feedparser**. The resulting entries are then normalized into a canonical **Document** data model. This normalization

process, defined in the `normalize_entry()` function, handles several key transformations:

- **URL Canonicalization:** The article’s primary link is extracted, with fallbacks to entry IDs, and cleaned of common tracking parameters.
- **Content Extraction:** Multiple possible content fields (e.g., `summary`, `description`, `content`) are evaluated, and the richest available text is selected and truncated to a configured maximum length.
- **Metadata Mapping:** Feed-level categories are mapped to the system’s internal tags, and publication dates are parsed from multiple possible fields (`published`, `updated`), defaulting to the current time if unavailable.
- **Source Confidence:** Each article is assigned an initial confidence score based on the configured confidence of its source.

This stage outputs a stream of standardized `Document` objects, ready for further processing, and logs detailed statistics on parsed entries, duplicates, and any parsing errors for monitoring.

4.2.2 Scraping subsystem, headless rendering and listing discovery

For sources without structured feeds, a headless web scraping subsystem was developed. It operates in two primary modes: **listing discovery** (finding article links on a homepage, listing page, or blog page) and **article extraction** (retrieving and parsing the full content of an individual article). A core design principle is the degradation from JavaScript-rendered to static HTML content as a fallback.

The scraping workflow is managed by several coordinated modules:

- **Fetcher (HTTPFetcher):** Handles all HTTP requests using `httpx`, incorporating exponential backoff retries, conditional GET requests (using ETag and Last-Modified headers), and configurable timeouts to be respectful to target servers.
- **Renderer (HeadlessRenderer):** Leverages `playwright` (v1.48.0) to execute JavaScript and render dynamic content. It includes optimizations such as blocking unnecessary resources (images, stylesheets, meta tags) to improve speed, and configurable wait conditions to ensure page stability before snapshot.
- **Listing Scraper (ListingScraper):** Discovers article URLs from a given page. It employs a multi-strategy approach, prioritizing structured metadata (JSON-LD, OpenGraph) before falling back to heuristic analysis of anchor tags (`<a>`). A filtering function, `is_probably_article_url()`, removes links to non-article pages (e.g., navigation, login, tags). Discovered URLs are canonicalized and deduplicated. It also respects the source’s `ignore_urls` list to filter out unwanted links.
- **Article Scraper (ArticleScraper):** Extracts the main content from a single article page. It invokes an LLM via an API call to extract fields like title and abstract, and directly asks it if the fetched page is an article or not, following a strict JSON schema.

The scraping strategy can be tailored per source via a `scrape_strategy` field (`js`, `static`, `auto`). The subsystem provides diagnostic output by logging fetch times, link counts, and error states.

4.3 Processing and Enrichment

4.3.1 Sanitization and Content Extraction

The first processing stage transforms raw, heterogeneous payloads from feeds and scrapers into a normalized, safe, and index-ready document format. This stage, implemented in the `sanitize_doc` function, is responsible for preparing fields for downstream enrichment and storage.

The sanitization process performs several operations:

- **Text Normalization:** All textual fields (e.g., `title`, `abstract`, `content`) are processed by a shared `_clean_str` helper. This function removes non-printable control characters, trims whitespace, and enforces configurable length limits defined by environment variables.
- **Date Handling:** Timestamps are parsed and normalized to ISO 8601 format using a utility function (`parse_to_iso8601`). The original publication timestamp string is preserved in a `published_at_raw` field for auditability.
- **Field Mapping:** The function maps and copies a defined set of fields from the input payload to the internal document model, including `url`, `source`, `language`, and user-defined `tags` and `categories`.
- **Data Preservation:** If an incoming article already contains a pre-computed `embedding` vector, this field is preserved to avoid redundant computation in later stages.

Content extraction for LLM classification occurs in a separate helper. To minimize context size and cost for LLM calls, this step fetches the page HTML statically, uses `BeautifulSoup` to strip scripts and styles, and extracts a concise text snippet along with structured metadata. An optional headless rendering step, controlled by configuration flags, is invoked based on heuristics such as link count and page structure, ensuring JavaScript-rendered content is captured when necessary.

4.3.2 LLM-Based Classification and Article Detection

The system employs a pattern-based LLM classification component (implemented in `llm_classifier.py`) to determine whether a fetched URL contains an article suitable for ingestion. This binary classification prevents non-article pages from progressing through the pipeline.

The classification process is structured as follows:

1. **Context Minimization:** The system performs a lightweight HTTP fetch and uses `BeautifulSoup` to extract a compact text snippet and metadata links from the page. This minimizes the context sent to the LLM, reducing token consumption and cost.
2. **Structured Prompting:** A predefined prompt template (`article_classification.md`) instructs the LLM to analyze the snippet and return a structured JSON response indicating if the page is an article and, if not, providing a concise reason.
3. **Robustness and Control:** Several mechanisms ensure the classifier's production readiness:

- **Rate Limiting and Caching:** A configurable token-bucket rate limiter (`LLM_RATE_LIMIT_PER_MIN`) prevents quota exhaustion. A Time-To-Live (TTL) cache stores recent classification decisions, eliminating redundant API calls for the same URL within a configurable window.
- **Heuristic Fallback:** If the LLM call times out, exceeds rate limits, or returns an error, the pipeline gracefully falls back to a deterministic heuristic rule-based classifier defined in `classify.py`, ensuring uninterrupted operation.
- **Auditability:** All LLM requests are logged with timing, token usage, and the outcome, supporting cost monitoring and debugging.

This classification pattern establishes the foundational scaffolding for more complex taxonomy assignments.

4.3.3 Metadata Enrichment and Abstract Generation

Following initial classification, articles undergo a metadata enrichment phase. The purpose is to derive structured, queryable fields from unstructured text, significantly enhancing search, filtering, and the intelligence value of the final document. This stage populates fields critical for the system’s domain-specific utility, including `abstract`, `sectors`, `criticality`, `tags`, and `offer`.

The enrichment pipeline is designed with the following principles:

- **LLM as a Structured Data Extractor:** The system leverages the LLM’s ability to follow instructions and output JSON. For each enrichment task (e.g., summarization, sector identification), a dedicated prompt template guides the LLM to extract the required information, adhering to a strict output schema. For instance, the `criticality` field is constrained to a controlled vocabulary (`low`, `medium`, `high`, `critical`), and `sectors` are mapped to a predefined taxonomy of 19 categories.
- **Idempotency and Determinism:** Where possible, enrichment steps are designed to be idempotent. Canonical operations like URL normalization and language detection produce the same output for a given input, ensuring stable updates during re-ingestion.
- **Provenance and Auditability:** The system maintains a clear lineage between source data and derived metadata. Original fields (`published_at_raw`, source-provided `abstract`) are preserved. LLM request logs for enrichment calls are retained, allowing for cost analysis and the ability to re-run or adjust enrichment logic without ever losing source data.

The final enrichment steps add user-centric metadata to improve content consumption.

Read-time estimation provides a practical indicator of article length. Using the `readtime` package, the helper function calculates reading minutes based on the sanitized `content` text, outputting a human-friendly string (e.g., “2 min read”).

If the ingested feed does not provide a suitable summary, the LLM is prompted to generate a concise `abstract` based on the article’s extracted content.

Operational safety is enforced through validation of LLM outputs against expected schemas and the use of configurable fallbacks. All enrichment steps are optional and controlled via feature flags, ensuring the pipeline remains functional even if external AI services experience downtime.

4.3.4 Embedding Computation, Deduplication, and Related-Article Linking

To enable semantic operations and content relationships, the pipeline computes a vector representation (embedding) for each article using the `sentence-transformers` library. The `EmbeddingComputer` class generates normalized vectors from the article's `title` and `abstract`, with the model (default: `all-MiniLM-L6-v2`) and execution device being configurable via environment variables.

These embeddings support two critical functions: deduplication and related-article linking. A two-tier deduplication strategy ensures data quality and idempotency. First, a deterministic SHA-1 hash of the article's `url` serves as the Elasticsearch `_id`, guaranteeing idempotent updates for the same URL. Second, a semantic similarity check compares embedding vectors via cosine similarity, flagging articles exceeding a configurable `RELATED_SIMILARITY_THRESHOLD` as potential near-duplicates.

The same embedding similarity mechanism powers the automated linking of related articles. The `RelatedArticlesHelper` queries recent articles with embeddings, calculates pairwise cosine similarities, and selects the top N candidates (configurable via `RELATED_MAX_ITEMS`) that meet the similarity threshold. The resulting `related` field, containing article IDs, is used by the API to provide contextual content recommendations.

4.4 Storage and Indexing

4.4.1 Index Design and Mapping Configuration

The system utilizes Elasticsearch as its search and retrieval engine, with a single index named `news` for storing processed articles. A central architectural principle is the separation of concerns: the ingestion pipeline does not write directly to Elasticsearch. Instead, it sends structured document batches to a dedicated FastAPI bulk endpoint. All mapping logic and write behavior is within the API layer, ensuring consistency, facilitating centralized validation, and allowing for safer evolution of the data schema without requiring changes to the ingestion codebase.

Each article is uniquely identified by a deterministic `_id` field, computed as a SHA-1 hash of its canonical URL. This approach provides inherent idempotency for writes, as repeated ingestion of the same article results in an update to the existing document rather than the creation of a duplicate.

The index mapping, defined in the `configure_es_index.py` utility, supports both efficient full-text search and complex aggregations. Design patterns include:

- **Dual-Purpose Text Fields:** Fields intended for search (`title`, `abstract`, `content`) are mapped as `text` type to enable language-aware analysis. Each is accompanied by a `.keyword` subfield of type `keyword` to support exact matching, sorting, and aggregations without tokenization.
- **Categorical Filtering with keyword:** Multi-valued fields used for filtering and faceting (`tags`, `sectors`, `categories`, `language`) are strictly typed as `keyword`. This guarantees consistent aggregation semantics and prevents the unexpected tokenization behavior of `text` fields.
- **Embedding Storage:** The `embedding` field is configured as a `dense_vector` type with a dimension of 384, matching the output of the `all-MiniLM-L6-v2` model. This enables efficient k-nearest neighbor (k-NN) searches for similarity-based deduplication and related article discovery directly within Elasticsearch, when the cluster license permits.

4.4.2 Bulk Ingestion and Error Handling Strategy

The system ingests articles in bulk to minimize the overhead of individual HTTP requests. The pipeline aggregates processed documents into configurable chunks, defined by the `BULK_MAX_DOCS` environment variable, before dispatching them to the API's `/articles/bulk` endpoint.

As described in Section 4.4.1, the deterministic `_id` ensures that re-ingesting the same article (e.g., from a subsequent pipeline cycle) results in an update rather than a duplicate. This is handled as an *upsert* operation within the bulk request, making the entire ingestion cycle safely repeatable.

Good error handling is critical for operational reliability. The API parses the Elasticsearch bulk response to identify the status of each individual document. Failures are categorized and handled according to their type:

- **Mapping/Validation Errors (e.g., 400):** Occur when a document violates the index mapping (e.g., an invalid date format for a `date` field). The offending document is logged with its error context and excluded from the index.
- **Transient Cluster Errors (429, 5xx):** Occur due to resource exhaustion or temporary cluster issues. The system implements an exponential backoff retry mechanism for the entire failed chunk. If retries are exhausted, a fallback strategy isolates problematic documents by re-attempting ingestion in progressively smaller batches or individually.

This multi-layered batching, deterministic idempotency, and granular error recovery ensures the pipeline is resilient to partial failures.

4.5 Backend API Service

The system's backend API is a FastAPI application serving as the central orchestration and data access layer. It encapsulates all business logic, data validation, and persistence operations, and provides a unified and secure interface for the pipeline and the Control Center. The application is structured for modularity, with clear separation between data models, API routes, and core business logic.

The service, located in the `apps/api` directory, is built around the following key components:

- **Data Models (models/):** `SQLModel` classes define the relational schema for entities such as `User`, `Source`, and `Email` in a SQLite database.
- **Request/Response Schemas (schemas/):** Pydantic models ensure consistent validation, serialization, and OpenAPI documentation for all endpoints, re-used across routers to maintain integrity.
- **Routers (routers/):** Endpoints are organized into modular routers (e.g., `auth.py`, `articles.py`, `sources.py`), each handling a specific domain. Complex business rules, such as article selection for emails, are delegated to dedicated helper modules (e.g., `core/article_selector.py`) to keep router logic concise.
- **Dependency Injection (core/deps.py):** Centralized dependencies manage authentication, database sessions, and Elasticsearch client acquisition, promoting code reuse and testability.

A dual-mode authentication system secures access to the API. For machine-to-machine communication, such as the pipeline worker ingesting new articles, a static API key (configured via the `API_STATIC_KEY` environment variable) is used. For human users interacting via the Control Center, JWT (JSON Web Token) authentication is employed. A common dependency, `allow_api_key_or_jwt`, allows specific endpoints to accept either credential type, while administrative endpoints restrict access to JWT tokens with an admin role.

A core architectural tenet is that the API is the sole writer to the primary data stores. The ingestion pipeline does not write directly to Elasticsearch or SQLite; instead, it calls the API's bulk ingestion endpoint (`POST /articles/bulk`).

4.6 Frontend Control Center

The Control Center is a React-based Single Page Application (SPA) that provides an administrative interface for the news pipeline. Built with TypeScript and Vite, it allows users to manage sources, explore processed articles, and configure automated email reports without requiring technical expertise.

The application's architecture emphasizes maintainability and a clean separation of concerns:

- **API Communication Layer** (`src/api/`): Dedicated modules (e.g., `articles.ts`, `auth.ts`) encapsulate all HTTP calls to the FastAPI backend. These modules are exported via a single index file, providing a unified interface for React components and centralizing request/response typing.
- **State and Authentication** (`src/auth/`): An `AuthContext` manages the user's login state, storing the JWT token in the browser's `localStorage` and providing it to API calls. Route protection is implemented via wrapper components (`ProtectedRoute`, `AdminRoute`) that redirect unauthenticated or unauthorized users.
- **Page Components** (`src/pages/`): The main user flows are organized into distinct pages:
 - **Dashboard**: Presents an overview with key metrics and time-series charts of incoming articles.
 - **Articles**: A feature-rich interface for browsing, filtering, and inspecting detailed article information, including related content and metadata.
 - **Sources**: Enables administrators to add, configure, enable, or disable news sources (RSS feeds and scrape targets).
 - **Email Composer**: Facilitates the creation and scheduling of personalized email reports, supporting both manual article selection and automatic selection based on embedding similarity.
 - **User Management**: An administrative page (exclusively for users with the `admin` role) for creating, editing, and deleting user accounts.

The frontend connects to an API instance using the `VITE_API_BASE` environment variable, set by the Docker Compose configuration, ensuring the bundled application communicates with the correct backend service.

4.7 Email Delivery

4.7.1 Article Selection for Email Reports

The article selection process is responsible for curating a relevant and non-redundant set of articles for inclusion in automated email reports. This process is executed on-demand when an email job is initiated, either manually via the Control Center or according to a schedule.

Selection begins by querying the canonical article store using a combination of filters and a ranking function. An initial filter removes articles older than a configurable threshold (e.g., 7 days) and applies deduplication logic to exclude near-identical content based on URL hashing and semantic similarity, as described in Section 4.3.4. A recipient-specific filter also prevents sending the same article to a user twice.

The core of the selection is a weighted scoring function that ranks remaining articles. This function combines multiple signals:

- **Freshness:** More recent articles receive a higher score.
- **Criticality:** Articles classified by the LLM as **high** or **critical** are boosted.
- **Relevance Match:** Articles whose **sectors** or **tags** intersect with the recipient’s defined interests receive a significant score increase.
- **Semantic Deduplication:** The system uses the pre-computed article embeddings to calculate cosine similarity against other articles, preventing two similar articles from being both selected.

Following ranking, a diversity pass ensures the final selection is not dominated by a single source or category, adhering to layout constraints such as a maximum number of articles per source. The system also supports editorial overrides, allowing administrators to pin mandatory articles or exclude specific items from a given send. The final output is an ordered list of article IDs, which is serialized and stored as part of the email job record, ensuring the selection is reproducible for auditing purposes.

4.7.2 Email Assembly and Delivery Mechanism

The email assembly and delivery subsystem transforms a list of selected articles into a formatted, personalized email and dispatches it to recipients.

The system uses HTML templates with defined placeholders for article metadata (title, abstract, URL, publication date) and recipient personalization tokens (e.g., `{recipient_name}`). The template engine performs token replacement and article data injection server-side, generating final HTML and a plain-text fallback.

For operational reliability, the system implements a retry mechanism with exponential backoff for transient provider failures. Each email dispatch is tracked as a discrete job with statuses (**queued**, **sending**, **sent**, **failed**). Detailed logs are captured and linked to the job. Furthermore, the fully rendered email content (HTML and plain text) is archived alongside the job record and the snapshot of selected article IDs. This archive provides a complete audit trail, allowing reconstruction of what was sent to any recipient at any given time.

5 Conclusion and Future Work

This report has presented the design, implementation, and core functionality of HORUS, a fully operational OSINT news ingestion and processing pipeline developed during a 16-week internship. The system successfully addresses the initial objectives by automating the collection of cybersecurity-relevant intelligence from diverse RSS/Atom feeds and websites, enriching articles with LLM-powered classification and summarization, and delivering curated content via an API, a React-based Control Center, and scheduled email reports.

The implemented pipeline demonstrates a practical application of modern AI-assisted content processing within a production-like environment. Technical contributions include an ingestion subsystem that handles structured feeds and JavaScript-rendered websites, a multi-stage processing pipeline featuring local vector embeddings for semantic deduplication and related-article linking, and a secure, API-first architecture. The integration of automated email reports, with its article selection and templated assembly, provides a direct, valuable output channel for the processed intelligence.

While the system is functionally complete and operational, its development involved several trade-offs and revealed areas for potential improvement. The reliance on external LLM services, while powerful, introduces variable latency and operational cost. The heuristic fallbacks and configurable timeouts provide resilience, but future work could explore more sophisticated offline models for classification to reduce dependency and cost. Similarly, the current Elasticsearch-based vector similarity, while effective, could be enhanced with dedicated ANN indexes if scale demands it.

5.1 Future Work

Several concrete avenues for future development and research emerge from this project:

- **Advanced User Profiles:** The current email selection uses recipient sectors of interest. A more sophisticated system could incorporate individual user preferences, such as including specific keywords to prioritize or exclude, preferred publication sources, or custom categories.
- **Multi-modal Enrichment:** Extending the pipeline to process images and attached documents (e.g., PDF reports often linked in articles) would increase the quality of extracted intelligence and improve user experience.
- **Real-time Alerting:** Implementing a real-time notification subsystem for high-criticality articles would complement the scheduled email reports.
- **Enhanced Source Reliability Scoring:** Moving beyond static confidence scores to a dynamic reputation system that adjusts source weighting based on the accuracy, timeliness, and uniqueness of the content it provides would improve overall data quality.
- **Operational Analytics Dashboard:** Expanding the Control Center with detailed operational dashboards showing pipeline health, LLM cost breakdowns, source performance metrics, and email engagement analytics would give administrators deeper insights.

Acknowledgements

I would like to express my sincere gratitude to Manuel Neto and Pedro Barbosa, my supervisors at Skill & Reach, for their invaluable guidance, support, and trust throughout this internship. I also extend my thanks to Professor Eduardo Marques, my academic supervisor, for their feedback during the development of this project. Finally, I wish to acknowledge the entire team at Skill & Reach for providing a collaborative and stimulating environment in which to work.